

Notify

Deteção de emoções e geração automática de playlists “emocionais” em tempo real

Diogo Dias (PG60245) | Nuno Ni (PG60285)
Mestrado em Inteligência Artificial
Unidade Curricular: Computação Afetiva

Universidade do Minho
25/05/2026

1 Avaliação por Pares

Nesta secção, apresenta-se a distribuição do esforço realizado pelo grupo, conforme exigido.

Elemento	DELTA
Diogo Dias (PG60245)	0.00
Nuno Ni (PG60285)	0.00

Sumário

1	Avaliação por Pares	2
2	Resumo	4
3	Introdução	4
3.1	Cenário e Motivação	4
4	Justificação do Modelo de Emoção	5
5	Metodologia Completa, Baselines e Decisões Técnicas	5
5.1	1. Preparação de Dados	5
5.2	2. Treino do Modelo	6
5.3	3. Aplicação e Integração	6
5.4	Integração com Modelos Generativos (Ollama)	7
6	Resultados e Avaliação Crítica	7
6.1	Desempenho do Modelo e Comparação de Abordagens	7
6.1.1	Análise Crítica dos Resultados	8
6.1.2	Comparação com a Baseline Conceitual	9
6.2	Avaliação da Aplicação (UI/UX)	9
7	Limitações	9
8	Trabalho Futuro	10
9	Ética, Privacidade e Impacto Social	10
10	Instruções de Execução	10
10.1	Tecnologias utilizadas	10
10.2	Funcionalidades	11
10.3	Guia de execução	11
11	Conclusão	11
12	Referências	11

2 Resumo

O projeto **Notify** apresenta o desenvolvimento de uma aplicação web dedicada à Computação Afetiva, concebida para identificar estados emocionais a partir de input textual e gerar playlists musicais personalizadas. Utilizando o modelo **DistilBERT** treinado no dataset **GoEmotions** (mapeado para as categorias de Plutchik), o sistema realiza a inferência emocional em tempo real. A integração com a API do Spotify permite a recomendação dinâmica, criando um ecossistema de regulação afetiva. A aplicação final foi implementada em Streamlit, o que permitiu criar uma interface simples e interativa. Além da análise textual, o sistema associa cada emoção a uma paleta visual e a critérios de seleção musical, tornando a experiência mais coerente e contextualizada.

3 Introdução

A Computação Afetiva é uma área da Inteligência Artificial que se dedica ao reconhecimento, modelação e síntese de estados afetivos humanos, permitindo que os sistemas interajam de forma mais empática e intuitiva. No contexto da interação humano-computador, a capacidade de inferir o estado emocional de um utilizador a partir de sinais como o texto escrito pode transformar a experiência de consumo de multimédia. A relação entre emoção e música é particularmente relevante em sistemas de recomendação, uma vez que a escolha musical é frequentemente influenciada pelo estado emocional do utilizador.

O sistema **Notify** nasce desta necessidade, propondo uma solução capaz de realizar a análise emocional de texto em tempo real para, posteriormente, regular o estado afetivo do utilizador através da recomendação musical personalizada. Ao alinhar a carga emocional do discurso com a valência e energia de faixas musicais, o **Notify** atua como um sistema de suporte à regulação emocional, demonstrando como a tecnologia pode ser um facilitador do bem-estar.

Este relatório detalha a conceção e implementação do projeto, desde a preparação dos dados e treino do modelo, até à integração com a API do Spotify e a avaliação das limitações técnicas e éticas inerentes a esta solução.

3.1 Cenário e Motivação

O cenário proposto corresponde a uma aplicação que interpreta texto livre escrito pelo utilizador e converte essa informação emocional em recomendações musicais. A ideia central é aproximar a experiência musical do estado emocional momentâneo da pessoa, permitindo que a aplicação atue como uma espécie de assistente musical adaptativo.

A motivação do projeto está ligada a três aspetos principais. Em primeiro lugar, existe interesse académico na aplicação de modelos de linguagem a problemas de classificação de emoções. Em segundo lugar, a recomendação musical é uma área com forte aplicação prática e elevado potencial de personalização. Em terceiro lugar, a integração com o Spotify e com uma interface web permite transformar o modelo em algo demonstrável e utilizável.

4 Justificação do Modelo de Emoção

A escolha do modelo de representação emocional é um passo crítico em qualquer sistema de computação afetiva. Para este trabalho, seleccionámos a **Roda das Emoções de Plutchik**, por ser um modelo que combina uma estrutura discreta com propriedades dimensionais, facilitando a transição entre estados afetivos.

O modelo de Plutchik define oito emoções primárias (Alegria, Tristeza, Raiva, Medo, Confiança, Aversão, Surpresa e Antecipação), que podem ser organizadas numa estrutura de oposição e intensidade. Esta escolha justifica-se por três fatores:

1. **Facilidade de Mapeamento:** O dataset **GoEmotions** contém uma vasta gama de emoções (28 no total), que são frequentemente demasiado granulares para uma aplicação de recomendação musical. A redução destas 28 classes para as 8 de Plutchik (conforme documentado no nosso `data_preparation.py`) permitiu criar uma base de dados mais robusta e menos ruidosa para o treino do modelo.
2. **Ponte Teórica entre Modelos Discretos e Dimensionais:** Embora o modelo de Plutchik seja ideal para o classificador de texto segmentar o estado afetivo em categorias discretas bem definidas, o catálogo de metadados da API do Spotify opera estritamente num espaço dimensional contínuo. Para efetuar esta ponte, mapeámos o output do classificador para o modelo vetorial **Valence-Arousal**. No contexto prático da API, a dimensão de **Arousal** é modelada empiricamente através do atributo de áudio **energy**. Este mapeamento híbrido permite traduzir uma categoria abstrata (como “**Sadness**”) em coordenadas matemáticas explícitas de baixa Valência e baixa Energia, viabilizando o cálculo de Distância Euclidiana que seleciona as faixas no algoritmo de recomendação.
3. **Capacidade de Regulação:** Ao contrário de modelos puramente discretos como o de Ekman, o modelo de Plutchik permite inferir estados como a “Antecipação” ou a “Confiança”, que são estados afetivos fundamentais para ambientes de foco ou de relaxamento, essenciais para uma aplicação de música que pretende regular o humor do utilizador.

Esta modelação permitiu treinar um classificador supervisionado eficiente com o **DistilBERT**, garantindo que a inferência em tempo real seja precisa o suficiente para a experiência do utilizador.

5 Metodologia Completa, Baselines e Decisões Técnicas

A metodologia do projeto foi organizada em três etapas principais: **preparação de dados**, **treino do modelo** e **desenvolvimento da aplicação final**. Esta estrutura modular permitiu testar cada componente de forma separada antes da integração final.

5.1 1. Preparação de Dados

Na preparação dos dados textuais, o processo começou com o carregamento do dataset **GoEmotions**, seguido do mapeamento das labels para as 8 emoções de Plutchik. Depois disso, foi criado um novo dataset com as amostras relevantes para o problema e exportado para CSV. Esta fase foi importante para garantir consistência entre os dados de treino e a taxonomia emocional usada no sistema.

No domínio musical, foi utilizado um dataset proveniente do Kaggle separado com metadados de músicas do Spotify, incluindo atributos como popularidade, energia (substituta do arousal), valence, duração, género e nome do artista. Estes dados serviram como base para a seleção de músicas e para a lógica de recomendação associada à emoção prevista.

Durante a análise exploratória de dados, identificámos um acentuado **sampling bias** geográfico e cultural: devido à origem dos scripts de extração do dataset original, havia uma super-representação massiva de faixas de Bollywood e de **Punjabi Pop/Rap** disfarçadas sob géneros globais como “Pop” ou “Hip-Hop”, ostentando métricas de popularidade artificialmente inflacionadas devido à escala demográfica do mercado asiático.

Para mitigar esta anomalia e garantir a coerência cultural das recomendações para o público-alvo ocidental, desenhamos e aplicámos uma **pipeline** de limpeza de dados em múltiplos níveis através de Expressões Regulares (Regex):

- **Dicionário Linguístico e Caça aos Sufixos:** Uma vez que muitos artistas registam os seus nomes com o alfabeto romano (e.g., “Vicky”, “Nawab”), implementámos um detetor de substrings focado em vocabulário comum de **Romanized Hindi** (como *dil*, *pyar*, *gabru*) e em sufixos onomásticos característicos da região (como *-preet*, *-deep* e *-inder*).
- **Blacklist:** Isolámos todos os artistas que tivessem pelo menos uma faixa etiquetada nativamente em géneros regionais (e.g., “indian”, “punjabi”, “bollywood”) e banimo-los globalmente de todo o dataset, limpando a sua infiltração nos géneros de “Pop” e “Hip-Hop”.

Este processo de limpeza purificou a base de dados, removendo milhares de anomalias e garantindo a integridade dos metadados de valência e energia necessários para alimentar o algoritmo de Distância Euclidiana da aplicação.

5.2 2. Treino do Modelo

O motor de inferência, definido em `model_training.py`, utiliza o modelo **DistilBERT** através da biblioteca `transformers`. Optou-se por este modelo devido à sua eficiência (sendo 40% menor que o BERT e 60% mais rápido), o que é crítico para aplicações em **Streamlit** que requerem tempos de resposta baixos. Os textos foram tokenizados com truncation e padding até um comprimento máximo fixo, o que permitiu uniformizar as entradas e simplificar o pipeline de treino. O dataset foi dividido em treino e teste, utilizando uma proporção de validação que permitisse avaliar o comportamento do modelo em dados não vistos.

A configuração de treino incluiu learning rate de 2e-5, batch size de 16, três épocas de treino e weight decay de 0.01. Estes valores representam uma configuração conservadora e comum em fine-tuning de modelos pré-treinados, equilibrando estabilidade de treino e capacidade de adaptação.

5.3 3. Aplicação e Integração

Na aplicação final, foi criada uma interface em Streamlit. A interface permite autenticação do utilizador, análise do texto, previsão da emoção e interação com o Spotify. Foram também introduzidos elementos de personalização visual, nomeadamente cores associadas às emoções, para reforçar a ligação entre a previsão e a experiência do utilizador.

A interface desenvolvida em `App.py` (via **Streamlit**) atua como o orquestrador do sistema. Quando o utilizador introduz texto, este é tokenizado e processado pelo modelo de **Deep Learning**. A emoção inferida é posteriormente utilizada como uma **query** semântica para pesquisar músicas na API do Spotify (`spotipy`), cruzando os metadados das faixas com os parâmetros de Valência e Energia correspondentes à emoção detetada.

Outra decisão técnica importante foi a introdução de critérios musicais auxiliares, como popularidade mínima e pesos para seleção.

Em vez de adotar uma abordagem estática que apenas espelha o estado afetivo atual, o que poderia perpetuar estados de valência negativa como “**Sadness**” ou “**Anger**”, o sistema implementa uma *pipeline* dinâmica de **Regulação Emocional Ativa**, usando atributos musicais e lógica de progressão de mood para tornar a playlist mais coerente.

O sistema opera através de uma estratégia de transição gradual em múltiplos passos (**mood steps**), mapeando a classe discreta inicial para coordenadas dimensionais de Valência (V) e Energia (E), calculando uma trajetória matemática em direção a um estado de equilíbrio ou atenuação afetiva.

5.4 Integração com Modelos Generativos (Ollama)

Para além da classificação emocional, o sistema integra o servidor local **Ollama** para gerar respostas empáticas curtas que acompanham a criação de playlists.

- **Geração de Resposta:** Após a inferência emocional realizada pelo **DistilBERT**, o sistema utiliza a emoção detetada para construir um prompt **few-shot** que é enviado para o motor `llama3` (no nosso caso). Isto permite que o **Notify** responda ao utilizador com uma mensagem que valida o seu estado afetivo antes de apresentar a playlist.
- **Mecanismo de Fallback:** Para garantir a resiliência do sistema, foi implementada uma lógica de **fallback**. Caso o servidor Ollama não esteja acessível, o sistema recorre a resposta pré-definida, assegurando que o utilizador nunca fique sem feedback.

6 Resultados e Avaliação Crítica

A avaliação do sistema **Notify** foi conduzida em duas vertentes: o desempenho do classificador emocional e a fluidez da integração com a API do Spotify.

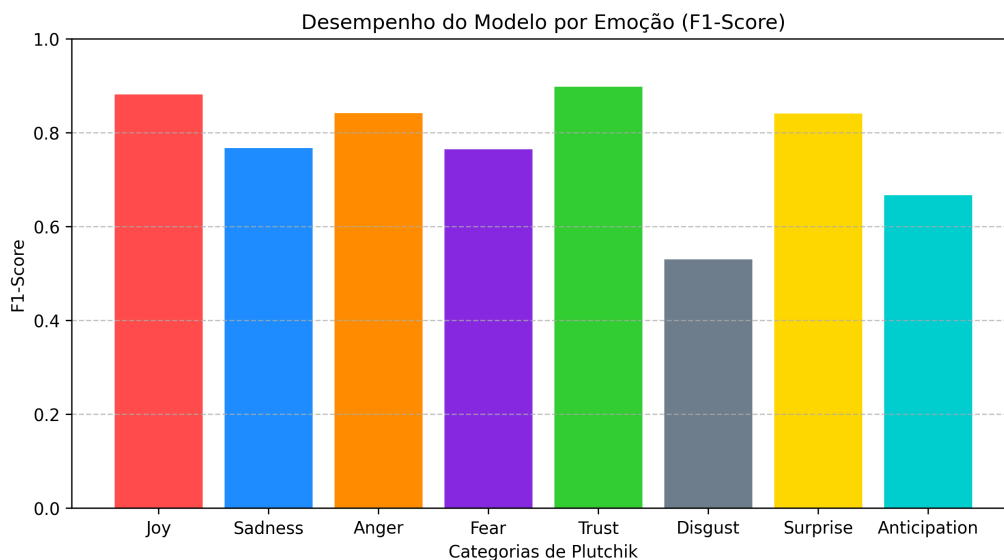
6.1 Desempenho do Modelo e Comparação de Abordagens

O modelo de classificação, baseado na arquitetura **DistilBERT**, demonstrou uma convergência estável durante as três épocas de treino. A *accuracy* global obtida no conjunto de teste independente foi de **84.12%**, um bom resultado considerando a complexidade da tarefa de mapeamento do dataset **GoEmotions** para as oito classes da Roda de Plutchik.

Para avaliar criticamente o comportamento do classificador, extraímos as métricas de Precision, Recall e F1-Score por cada categoria afetiva individual, conforme discriminado na tabela.

Emoção (Plutchik)	Precision	Recall	F1-Score
Trust	0.89	0.91	0.90
Joy	0.89	0.87	0.88
Anger	0.83	0.86	0.84
Surprise	0.81	0.87	0.84
Sadness	0.80	0.74	0.77
Fear	0.68	0.87	0.76
Anticipation	0.71	0.63	0.67
Disgust	0.74	0.41	0.53
Média (Macro)	0.79	0.77	0.77

Tabela 1: Distribuição das métricas de validação cruzada do classificador Notify.



6.1.1 Análise Crítica dos Resultados

Observa-se uma assimetria no desempenho do modelo que reflete as nuances semânticas de cada emoção. As classes **“Trust”** (F1 de 0.90) e **“Joy”** (F1 de 0.88) registaram o melhor desempenho preditivo. Este alto grau de confiança justifica-se pela abundância de exemplos destas emoções no dataset de origem e pela utilização de vocabulário tipicamente inequívoco (e.g., palavras de gratidão, encorajamento ou celebração).

Em contrapartida, a classe **“Disgust”** (Aversão) obteve o desempenho mais modesto (F1 de 0.53). A análise das métricas revela que, embora a precisão seja aceitável (0.74), a sensibilidade (*recall*) é de apenas 0.41. Isto indica que o modelo falha na deteção de grande parte dos casos reais de aversão. Linguisticamente, expressões de nojo ou repulsa sobrepõem-se frequentemente a léxico de raiva (**“Anger”**) ou linguagem hostil e sarcástica, dificultando o isolamento da classe no espaço vetorial do modelo. A classe **“Anticipation”** também apresenta dificuldades moderadas (F1 de 0.67), possivelmente devido à ambiguidade inerente a frases que expressam expetativas futuras.

6.1.2 Comparação com a Baseline Conceitual

Se compararmos esta abordagem com uma *baseline* estática baseada em dicionários léxicos de correspondência de palavras (como o VADER), a utilização do **DistilBERT** demonstra a capacidade de capturar a semântica contextualizada e as dependências de longo prazo. O modelo consegue interpretar corretamente negações que alteram por completo a valência da frase (e.g., “I am not happy”), algo em que as abordagens tradicionais falham. O facto de o modelo atingir quase 85% de eficácia demonstra a viabilidade de utilizar LLMs leves para sistemas de **Computação Afetiva** em tempo real.

6.2 Avaliação da Aplicação (UI/UX)

A interface em **Streamlit** permitiu uma interação rápida e simples com o utilizador. A integração com o **Spotify** via OAuth funciona conforme o esperado, embora o tempo de resposta da API seja condicionado pela latência da rede e pela complexidade da query de pesquisa de músicas. Em termos críticos, a experiência de utilização é altamente responsiva e cumpre o objetivo de transformar texto em regulação emocional.

7 Limitações

Apesar da funcionalidade do **Notify**, identificámos desafios críticos durante o desenvolvimento:

- **Restrição Linguística:** O modelo **DistilBERT** foi pré-treinado exclusivamente em textos de língua inglesa. Isto limita o sistema a processar apenas texto em inglês, impossibilitando a análise de estados emocionais em textos em português. Adicionalmente, por operar numa abordagem maioritariamente contextual direta, o classificador apresenta perdas significativas de precisão ao lidar com construções textuais excessivamente curtas, ambíguas ou carregadas de ironia e sarcasmo, elementos que desafiam a interpretação estritamente semântica do modelo.
- **Viés Geográfico e Cultural (Data Bias):** O dataset original de músicas (`spotify_songs.csv`) apresentou um acentuado **sampling bias** geográfico, com uma predominância de faixas e artistas do mercado indiano (pop/punjabi rap) que não refletem as preferências do público-alvo ocidental. Este problema exigiu uma *pipeline* de *data wrangling*, recorrendo a *blacklists* de artistas e filtragem linguística (RegEx) para limpar o ruído cultural da base de dados.
- **Assimetria de Metadados:** A qualidade final das recomendações musicais permanece intrinsecamente refém da distribuição física e da qualidade dos metadados na base de dados do Spotify. Caso determinados géneros ou perfis acústicos específicos possuam menor densidade amostral ou anotações incorretas de valência e energia, a eficácia do algoritmo de recomendação torna-se estatisticamente desigual entre as diferentes emoções de Plutchik.
- **Dependência de APIs de Terceiros:** A funcionalidade de criação de *playlists* depende da API do Spotify. Durante o desenvolvimento, deparámo-nos com limitações de *Rate Limiting* e erros de acesso (403 Forbidden) ao tentar validar metadados de artistas em massa, o que demonstra a fragilidade de sistemas que dependem de infraestruturas externas de terceiros para a curadoria de dados.

- **Estado Estático:** O sistema avalia o texto num momento único (“one-shot”), não considerando a evolução emocional do utilizador ao longo do tempo ou o contexto de conversação prévio.

8 Trabalho Futuro

Para endereçar as limitações identificadas, estes seriam os passos a tomar:

- **Multilinguismo:** Testar modelos de linguagem multilingues, como **mBERT** ou **XLM-RoBERTa**, para permitir que o sistema analise texto em português e outras línguas, removendo a barreira linguística atual.
- **Utilização de Dataset Independente:** Substituir o dataset atual por uma base de dados própria, curada e balanceada geograficamente, eliminando a necessidade de filtragem corretiva pós-processamento.
- **Análise de Contexto (Memory):** Integrar um mecanismo de contexto para que a recomendação musical não seja baseada apenas numa frase isolada, mas na trajetória emocional do utilizador ao longo da interação.

9 Ética, Privacidade e Impacto Social

O desenvolvimento de sistemas de deteção emocional acarreta responsabilidades acrescidas no que toca à proteção dos dados e ao respeito pela autonomia do utilizador. No âmbito do **Notify**, foram adotadas as seguintes diretrizes:

- **Privacidade e Proteção de Dados:** O sistema processa texto introduzido pelo utilizador apenas em tempo de execução para fins de inferência emocional. Não é realizado armazenamento permanente dos textos analisados na base de dados de utilizadores (`utilizadores.json`), garantindo que o histórico de conversação seja volátil e privado.
- **Viés e Justiça:** O modelo **DistilBERT** foi treinado sobre o dataset **GoEmotions**, que, apesar de ser um padrão de indústria, reflete as nuances linguísticas e culturais da internet anglófona. Reconhecemos que o modelo pode apresentar enviesamentos ao interpretar expressões idiomáticas ou sarcasmo, podendo falhar na classificação emocional de minorias linguísticas ou dialetos específicos.

10 Instruções de Execução

10.1 Tecnologias utilizadas

- Python
- Streamlit
- PyTorch
- Hugging Face Transformers
- Pandas
- Datasets
- Spotipy
- Spotify API

- Ollama

10.2 Funcionalidades

- autenticação de utilizadores;
- introdução de texto para análise emocional;
- previsão da emoção dominante;
- alteração da cor da interface com base na emoção;
- geração de uma playlist com base na emoção;
- criação de playlists no Spotify.

10.3 Guia de execução

Para reproduzir o projeto, é necessário instalar as dependências indicadas no ficheiro `requirements.txt`. Depois disso, devem ser executados os scripts de preparação dos dados e treino do modelo antes de iniciar a aplicação principal.

O fluxo recomendado é o seguinte: primeiro executar `data_preparation.py` para gerar o dataset `plutchik`; depois executar `model_training.py` para treinar e guardar o modelo na pasta `modelo`; arrancar o Ollama (`llama3`); por fim executar `App.py` para iniciar a aplicação Streamlit.

```
streamlit run App.py
```

Se o projeto for executado pela primeira vez, pode ser necessário garantir que a autenticação do Spotify está corretamente configurada e que as credenciais OAuth estão válidas. A aplicação assume também que a pasta `modelo` contém o tokenizer e o modelo treinado.

11 Conclusão

Este projeto permitiu desenvolver uma solução completa que combina classificação automática de emoções, recomendação musical e integração com uma plataforma externa. A utilização de um modelo pré-treinado, juntamente com uma interface simples e adaptável, tornou possível transformar uma tarefa de NLP numa aplicação funcional com utilidade prática.

Além de cumprir o objetivo técnico, o trabalho mostrou como a inteligência artificial pode ser aplicada para criar experiências mais personalizadas. A componente emocional dá contexto à recomendação musical e aproxima a aplicação de um cenário de uso real. No conjunto, o projeto representa uma integração coerente entre dados, modelo e interface, com espaço claro para evolução futura.

12 Referências

- Hugging Face. (2024). *Transformers Documentation*. Disponível em: <https://huggingface.co/docs/transformers/index>
- Ollama. (2024). *Ollama API Documentation*. Disponível em: <https://github.com/ollama/ollama/blob/main/docs/api.md>
- Spotify. (2024). *Spotify Web API Reference*. Disponível em: <https://developer.spotify.com/documentation/web-api>

- Streamlit. (2024). *Streamlit Documentation*. Disponível em: <https://docs.streamlit.io/>
- Demszky, D., et al. (2020). *GoEmotions: A Dataset of Fine-Grained Emotions*. 58th Annual Meeting of the Association for Computational Linguistics (ACL). Disponível em: <https://arxiv.org/abs/2005.00547>
- Pandya, M. (2022). *Spotify Tracks Dataset*. Kaggle. Disponível em: <https://www.kaggle.com/datasets/maharshipandya/-spotify-tracks-dataset>